Vous : Voici mon code : import sys

```
import os
from PyQt6.QtWidgets import (QApplication, QWidget, QVBoxLayout, QHBoxLayout,
                             QPushButton, QFileDialog, QLabel, QMessageBox,
                             QFrame, QSizePolicy, QSpacerItem)
from PyQt6.QtCore import Qt
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms


class LogicielCryptage(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("SecureFolder - Streaming Edition")
        self.resize(550, 400) # Légèrement agrandi pour le texte
        self.setMinimumSize(450, 300)

        self.chemin_cible = None
        self.cle = None
        self.fichier_cle = "ma_cle_chacha20.key"
        self.taille_chunk = 64 * 1024  # 64 Ko par morceau

        self.initUI()
```

```
    self.appliquer_style()
    self.charger_ou_generer_cle()

def initUI(self):
    layout_principal = QVBoxLayout()
    layout_principal.setContentsMargins(30, 30, 30, 30)
    layout_principal.setSpacing(20)

    # --- CORRECTION DU TEXTE ICI ---
    texte_titre = (
        "Avec le ChaCha20 pur, un fichier individuel à l'intérieur\n"
        "du dossier peut parfaitement dépasser 256 Go.\n"
        "Aucun Message d'Erreur si un Fichier dépasse 256 Go !"
    )
    titre = QLabel(texte_titre)
    titre.setObjectName("titre")
    titre.setAlignment(Qt.AlignmentFlag.AlignCenter)
    titre.setWordWrap(True) # Permet au texte de s'adapter à la fenêtre
    layout_principal.addWidget(titre)

    frame_fichier = QFrame()
```

```
frame_fichier.setObjectName("frameFichier")
layout_fichier = QVBoxLayout(frame_fichier)

self.label_info = QLabel("Aucun dossier sélectionné")
self.label_info.setAlignment(Qt.AlignmentFlag.AlignCenter)
self.label_info.setWordWrap(True)

self.btn_choisir = QPushButton("Parcourir les dossiers...")
self.btn_choisir.setObjectName("btnChoisir")
self.btn_choisir.setCursor(Qt.CursorShape.PointingHandCursor)
self.btn_choisir.clicked.connect(self.choisir_dossier)

layout_fichier.addWidget(self.label_info)
layout_fichier.addWidget(self.btn_choisir, alignment=Qt.AlignmentFlag.AlignCenter)
layout_principal.addWidget(frame_fichier)

layout_principal.addSpacerItem(QSpacerItem(20, 20, QSizePolicy.Policy.Minimum, QSizePolicy.Policy.Expanding))

layout_boutons = QHBoxLayout()
layout_boutons.setSpacing(15)
```

```
    self.btn_chiffrer = QPushButton("🔒 Chiffrer le dossier")
    self.btn_chiffrer.setObjectName("btnAction")
    self.btn_chiffrer.clicked.connect(self.chiffrer_dossier)

    self.btn_dechiffrer = QPushButton("🔓 Déchiffrer le dossier")
    self.btn_dechiffrer.setObjectName("btnAction")
    self.btn_dechiffrer.clicked.connect(self.dechiffrer_dossier)

    self.btn_chiffrer.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)
    self.btn_dechiffrer.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)

    layout_boutons.addWidget(self.btn_chiffrer)
    layout_boutons.addWidget(self.btn_dechiffrer)

    layout_principal.addLayout(layout_boutons)
    self.setLayout(layout_principal)

def appliquer_style(self):
    style = """
        QWidget { background-color: #f4f5f7; font-family: 'Segoe UI', Arial, sans-serif; color: #333333; }
        QLabel#titre { font-size: 16px; font-weight: bold; color: #2980b9; margin-bottom: 10px; }
```

```
        QFrame#frameFichier { background-color: #ffffff; border: 2px dashed #bdc3c7; border-radius: 10px; padding: 20px; }
        QLabel { font-size: 14px; color: #7f8c8d; }
        QPushButton { font-size: 14px; font-weight: bold; padding: 10px 20px; border-radius: 6px; border: none; color: white; }
        QPushButton#btnChoisir { background-color: #e74c3c; margin-top: 10px; }
        QPushButton#btnChoisir:hover { background-color: #2980b9; }
        QPushButton#btnChoisir:pressed { background-color: #922b21; }
        QPushButton#btnAction { background-color: #2ecc71; padding: 12px; font-size: 15px; }
        QPushButton#btnAction:hover { background-color: #27ae60; }
        QPushButton#btnAction:pressed { background-color: #1e8449; }
        QPushButton:disabled { background-color: #bdc3c7; color: #7f8c8d; }
    """
    self.setStyleSheet(style)

def charger_ou_generer_cle(self):
    if not os.path.exists(self.fichier_cle):
        self.cle = os.urandom(32)
        with open(self.fichier_cle, "wb") as f_cle:
            f_cle.write(self.cle)
    else:
        with open(self.fichier_cle, "rb") as f_cle:
            self.cle = f_cle.read()
```

```
def choisir_dossier(self):
    chemin = QFileDialog.getExistingDirectory(self, "Sélectionner un dossier")
    if chemin:
        self.chemin_cible = chemin
        nom_dossier = os.path.basename(chemin)
        self.label_info.setText(f"<b>Dossier prêt :</b> {nom_dossier}")
        self.label_info.setStyleSheet("color: #2c3e50;")

def chiffrer_dossier(self):
    if not self.chemin_cible:
        QMessageBox.warning(self, "Erreur", "Veuillez d'abord choisir un dossier.")
        return

    # Sécurité : on désactive les boutons pendant le travail
    self.btn_chiffrer.setEnabled(False)
    self.btn_dechiffrer.setEnabled(False)
    self.btn_choisir.setEnabled(False)
    QApplication.processEvents()

    fichiers_traites = 0
```

```
erreurs = 0

for racine, sous_dossiers, fichiers in os.walk(self.chemin_cible):
    for nom_fichier in fichiers:
        if nom_fichier.endswith(".enc") or nom_fichier == self.fichier_cle:
            continue

        chemin_complet = os.path.join(racine, nom_fichier)
        chemin_chiffre = chemin_complet + ".enc"

        try:
            nonce = os.urandom(16)
            cipher = Cipher(algorithms.ChaCha20(self.cle, nonce), mode=None)
            encryptor = cipher.encryptor()

            with open(chemin_complet, "rb") as f_in, open(chemin_chiffre, "wb") as f_out:
                f_out.write(nonce)

                while True:
                    chunk = f_in.read(self.taille_chunk)
                    if not chunk:
```

```
                break
            f_out.write(encryptor.update(chunk))

        os.remove(chemin_complet)
        fichiers_traites += 1
        QApplication.processEvents()

    except Exception as e:
        erreurs += 1
        print(f"Erreur de chiffrement sur {nom_fichier} : {e}")

# On réactive les boutons
self.btn_chiffrer.setEnabled(True)
self.btn_dechiffrer.setEnabled(True)
self.btn_choisir.setEnabled(True)

QMessageBox.information(self, "Terminé", f"{fichiers_traites} fichiers chiffrés.\n{erreurs} erreurs rencontrées.")

def dechiffrer_dossier(self):
    if not self.chemin_cible:
        QMessageBox.warning(self, "Erreur", "Veuillez d'abord choisir un dossier.")
```

```
    return

# Sécurité : on désactive les boutons pendant le travail
self.btn_chiffrer.setEnabled(False)
self.btn_dechiffrer.setEnabled(False)
self.btn_choisir.setEnabled(False)
QApplication.processEvents()

fichiers_traites = 0
erreurs = 0

for racine, sous_dossiers, fichiers in os.walk(self.chemin_cible):
    for nom_fichier in fichiers:
        if not nom_fichier.endswith(".enc"):
            continue

        chemin_complet = os.path.join(racine, nom_fichier)
        chemin_original = chemin_complet[:-4]

        try:
            with open(chemin_complet, "rb") as f_in, open(chemin_original, "wb") as f_out:
```

```
            nonce = f_in.read(16)
            cipher = Cipher(algorithms.ChaCha20(self.cle, nonce), mode=None)
            decryptor = cipher.decryptor()

            while True:
                chunk = f_in.read(self.taille_chunk)
                if not chunk:
                    break
                f_out.write(decryptor.update(chunk))

        os.remove(chemin_complet)
        fichiers_traites += 1
        QApplication.processEvents()

    except Exception as e:
        erreurs += 1
        print(f"Erreur de déchiffrement sur {nom_fichier} : {e}")

# On réactive les boutons
self.btn_chiffrer.setEnabled(True)
self.btn_dechiffrer.setEnabled(True)
```

```
        self.btn_choisir.setEnabled(True)

        QMessageBox.information(self, "Terminé", f"{fichiers_traites} fichiers déchiffrés.\n{erreurs} erreurs rencontrées.")

if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = LogicielCryptage()
    fenetre.show()
    sys.exit(app.exec())
```

Gemini a dit

Bonjour ! C'est un très beau projet. Votre code est propre, bien structuré, et l'interface PyQt6 est élégante avec sa feuille de style (CSS) intégrée.

Vous avez particulièrement bien géré le traitement des fichiers volumineux : l'utilisation de la lecture par blocs (*chunking* avec self.taille_chunk) est exactement la bonne approche pour éviter de saturer la mémoire RAM lors du chiffrement de fichiers de plusieurs gigaoctets.

Puisque vous avez partagé votre code sans question spécifique, je vous propose une petite revue de code avec **les points forts** et **quelques axes d'amélioration** pour rendre votre application plus professionnelle et sécurisée.

🌟 **Points forts de votre code**

- **Traitement en Streaming :** La boucle while True avec des morceaux de 64 Ko est parfaite pour les fichiers massifs.
- **Génération de Nonce aléatoire :** Utiliser un os.urandom(16) unique pour chaque fichier chiffré est une excellente pratique de cryptographie.

- **Expérience Utilisateur (UX) :** L'interface est claire, empêche les doubles-clics accidentels (désactivation des boutons) et informe l'utilisateur du résultat à la fin.

🛠️ Axes d'amélioration (Sécurité et Performance)

1. Intégrité des données (Très important en sécurité)

- **Le problème :** Vous utilisez le ChaCha20 "pur". C'est un excellent algorithme de chiffrement, mais il n'assure *pas* l'intégrité des données. Si un fichier .enc est légèrement corrompu ou modifié de manière malveillante, le déchiffrement se fera sans erreur, mais produira un fichier final corrompu et inutilisable.
- **La solution :** Il est fortement recommandé d'utiliser **ChaCha20Poly1305** (qui est de l'AEAD - *Authenticated Encryption with Associated Data*). Poly1305 ajoute une "étiquette" (un tag) de vérification. Lors du déchiffrement, si le fichier a été modifié même d'un seul octet, l'algorithme lèvera une erreur de sécurité au lieu de recracher des données corrompues.

2. Gestion de la clé de chiffrement

- **Le problème :** Votre script génère une clé et la sauvegarde en clair dans un fichier ma_cle_chacha20.key juste à côté du script. Si un pirate accède à l'ordinateur, il aura à la fois les fichiers chiffrés et la clé pour les lire.
- **La solution :** Au lieu de stocker la clé sur le disque, il serait plus sûr de demander un **mot de passe** à l'utilisateur via une petite boîte de dialogue (QInputDialog), puis de dériver ce mot de passe en une clé de 32 octets en utilisant une fonction de hachage robuste comme **PBKDF2** ou **Scrypt** (tous deux disponibles dans la bibliothèque cryptography).

3. Blocage de l'interface graphique (GUI)

- **Le problème :** Vous utilisez QApplication.processEvents() pour garder l'interface réactive pendant la boucle. Bien que cela fonctionne pour de petits dossiers, c'est considéré comme une mauvaise pratique en PyQt. Si l'utilisateur clique sur la croix pour fermer la fenêtre pendant le traitement, cela peut créer un crash ou laisser des fichiers à moitié chiffrés.
- **La solution :** Les opérations longues comme le chiffrement de gros dossiers devraient être placées dans un **QThread** séparé. Cela permet à l'interface de rester 100% fluide, et vous pourriez même ajouter une vraie barre de progression (QProgressBar) !

4. Finalisation du chiffrement

- **Le détail technique :** Bien que ChaCha20 soit un chiffrement de flux, la documentation de la bibliothèque cryptography recommande de toujours appeler .finalize() à la fin du traitement pour s'assurer que tout le tampon est vidé. Par exemple : f_out.write(encryptor.finalize()) après votre boucle while.

Souhaitez-vous que je vous montre comment implémenter l'une de ces améliorations, comme **l'ajout d'un** QThread **pour intégrer une barre de progression**, ou le passage à **ChaCha20Poly1305 / Mot de passe** pour renforcer la sécurité ?

Vous avez dit

ajout d'un QThread pour intégrer une barre de progression

Gemini a dit

Excellent choix ! C'est la modification qui aura le plus grand impact sur l'expérience utilisateur et la stabilité de votre application.

En utilisant un QThread, nous allons séparer l'interface graphique (le thread principal) du travail lourd de cryptographie (le thread secondaire). Les deux communiqueront via des **signaux** (pyqtSignal), ce qui est la méthode la plus sûre en PyQt.

Voici votre code mis à jour et fonctionnel avec une QProgressBar et un QThread.

Le code mis à jour

Python

```
import sys

import os

from PyQt6.QtWidgets import (QApplication, QWidget, QVBoxLayout, QHBoxLayout,

                             QPushButton, QFileDialog, QLabel, QMessageBox,

                             QFrame, QSizePolicy, QSpacerItem, QProgressBar)

from PyQt6.QtCore import Qt, QThread, pyqtSignal

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
```

```
# ======================================================================
# 1. CRÉATION DU WORKER (Le thread qui fait le travail lourd en arrière-plan)
# ======================================================================
class WorkerCryptage(QThread):
    # Définition des signaux pour communiquer avec l'interface graphique
    progression = pyqtSignal(int)
    info_fichier = pyqtSignal(str)
    termine = pyqtSignal(int, int) # (fichiers_traites, erreurs)

    def __init__(self, chemin_cible, cle, taille_chunk, action):
        super().__init__()
        self.chemin_cible = chemin_cible
        self.cle = cle
        self.taille_chunk = taille_chunk
        self.action = action # 'chiffrer' ou 'dechiffrer'

    def run(self):
        # Étape 1 : Lister tous les fichiers valides pour calculer le total
        fichiers_a_traiter = []
        for racine, sous_dossiers, fichiers in os.walk(self.chemin_cible):
```

```
    for nom_fichier in fichiers:
        if self.action == 'chiffrer':
            if not nom_fichier.endswith(".enc") and nom_fichier != "ma_cle_chacha20.key":
                fichiers_a_traiter.append(os.path.join(racine, nom_fichier))
        elif self.action == 'dechiffrer':
            if nom_fichier.endswith(".enc"):
                fichiers_a_traiter.append(os.path.join(racine, nom_fichier))

total_fichiers = len(fichiers_a_traiter)
if total_fichiers == 0:
    self.termine.emit(0, 0)
    return

fichiers_traites = 0
erreurs = 0

# Étape 2 : Traitement des fichiers
for index, chemin_complet in enumerate(fichiers_a_traiter):
    nom_fichier = os.path.basename(chemin_complet)
    self.info_fichier.emit(f"Traitement : {nom_fichier}")
```

```
try:
    if self.action == 'chiffrer':
        chemin_chiffre = chemin_complet + ".enc"
        nonce = os.urandom(16)
        cipher = Cipher(algorithms.ChaCha20(self.cle, nonce), mode=None)
        encryptor = cipher.encryptor()

        with open(chemin_complet, "rb") as f_in, open(chemin_chiffre, "wb") as f_out:
            f_out.write(nonce)
            while True:
                chunk = f_in.read(self.taille_chunk)
                if not chunk: break
                f_out.write(encryptor.update(chunk))
        os.remove(chemin_complet)

    elif self.action == 'dechiffrer':
        chemin_original = chemin_complet[:-4]
        with open(chemin_complet, "rb") as f_in, open(chemin_original, "wb") as f_out:
            nonce = f_in.read(16)
            cipher = Cipher(algorithms.ChaCha20(self.cle, nonce), mode=None)
            decryptor = cipher.decryptor()
```

```
                while True:
                    chunk = f_in.read(self.taille_chunk)
                    if not chunk: break
                    f_out.write(decryptor.update(chunk))
            os.remove(chemin_complet)

        fichiers_traites += 1

    except Exception as e:
        erreurs += 1
        print(f"Erreur sur {nom_fichier} : {e}")

    # Calcul et envoi du pourcentage de progression
    pourcentage = int(((index + 1) / total_fichiers) * 100)
    self.progression.emit(pourcentage)

self.termine.emit(fichiers_traites, erreurs)


# ======================================================================
```

```
# 2. L'INTERFACE GRAPHIQUE
# ======================================================================
class LogicielCryptage(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("SecureFolder - Streaming Edition")
        self.resize(550, 450) # Légèrement agrandi pour la barre
        self.setMinimumSize(450, 350)

        self.chemin_cible = None
        self.cle = None
        self.fichier_cle = "ma_cle_chacha20.key"
        self.taille_chunk = 64 * 1024

        self.initUI()
        self.appliquer_style()
        self.charger_ou_generer_cle()

    def initUI(self):
        layout_principal = QVBoxLayout()
        layout_principal.setContentsMargins(30, 30, 30, 30)
```

```
layout_principal.setSpacing(20)

texte_titre = (
    "Avec le ChaCha20 pur, un fichier individuel à l'intérieur\n"
    "du dossier peut parfaitement dépasser 256 Go.\n"
    "Aucun Message d'Erreur si un Fichier dépasse 256 Go !"
)
titre = QLabel(texte_titre)
titre.setObjectName("titre")
titre.setAlignment(Qt.AlignmentFlag.AlignCenter)
titre.setWordWrap(True)
layout_principal.addWidget(titre)

frame_fichier = QFrame()
frame_fichier.setObjectName("frameFichier")
layout_fichier = QVBoxLayout(frame_fichier)

self.label_info = QLabel("Aucun dossier sélectionné")
self.label_info.setAlignment(Qt.AlignmentFlag.AlignCenter)
self.label_info.setWordWrap(True)
```

```
self.btn_choisir = QPushButton("Parcourir les dossiers...")
self.btn_choisir.setObjectName("btnChoisir")
self.btn_choisir.setCursor(Qt.CursorShape.PointingHandCursor)
self.btn_choisir.clicked.connect(self.choisir_dossier)

layout_fichier.addWidget(self.label_info)
layout_fichier.addWidget(self.btn_choisir, alignment=Qt.AlignmentFlag.AlignCenter)
layout_principal.addWidget(frame_fichier)

# --- NOUVEAU : Barre de progression et label d'information en temps réel ---
self.label_progression = QLabel("")
self.label_progression.setAlignment(Qt.AlignmentFlag.AlignCenter)
self.label_progression.setStyleSheet("font-size: 12px; color: #7f8c8d;")
layout_principal.addWidget(self.label_progression)

self.progress_bar = QProgressBar()
self.progress_bar.setValue(0)
self.progress_bar.setTextVisible(True)
self.progress_bar.setFixedHeight(20)
layout_principal.addWidget(self.progress_bar)
# ----------------------------------------------------------------------
```

```
layout_principal.addSpacerItem(QSpacerItem(20, 10, QSizePolicy.Policy.Minimum, QSizePolicy.Policy.Expanding))

layout_boutons = QHBoxLayout()
layout_boutons.setSpacing(15)

self.btn_chiffrer = QPushButton("🔒 Chiffrer le dossier")
self.btn_chiffrer.setObjectName("btnAction")
self.btn_chiffrer.clicked.connect(lambda: self.lancer_travail('chiffrer'))

self.btn_dechiffrer = QPushButton("🔓 Déchiffrer le dossier")
self.btn_dechiffrer.setObjectName("btnAction")
self.btn_dechiffrer.clicked.connect(lambda: self.lancer_travail('dechiffrer'))

self.btn_chiffrer.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)
self.btn_dechiffrer.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)

layout_boutons.addWidget(self.btn_chiffrer)
layout_boutons.addWidget(self.btn_dechiffrer)

layout_principal.addLayout(layout_boutons)
```

```
    self.setLayout(layout_principal)


def appliquer_style(self):
    style = """
        QWidget { background-color: #f4f5f7; font-family: 'Segoe UI', Arial, sans-serif; color: #333333; }
        QLabel#titre { font-size: 16px; font-weight: bold; color: #2980b9; margin-bottom: 10px; }
        QFrame#frameFichier { background-color: #ffffff; border: 2px dashed #bdc3c7; border-radius: 10px; padding: 20px; }
        QLabel { font-size: 14px; color: #7f8c8d; }
        QPushButton { font-size: 14px; font-weight: bold; padding: 10px 20px; border-radius: 6px; border: none; color: white; }
        QPushButton#btnChoisir { background-color: #e74c3c; margin-top: 10px; }
        QPushButton#btnChoisir:hover { background-color: #2980b9; }
        QPushButton#btnChoisir:pressed { background-color: #922b21; }
        QPushButton#btnAction { background-color: #2ecc71; padding: 12px; font-size: 15px; }
        QPushButton#btnAction:hover { background-color: #27ae60; }
        QPushButton#btnAction:pressed { background-color: #1e8449; }
        QPushButton:disabled { background-color: #bdc3c7; color: #7f8c8d; }
        QProgressBar { border: 1px solid #bdc3c7; border-radius: 5px; text-align: center; color: black; font-weight: bold; background-color: #ecf0f1; }
        QProgressBar::chunk { background-color: #3498db; border-radius: 4px; }
    """
    self.setStyleSheet(style)
```

```
def charger_ou_generer_cle(self):
    if not os.path.exists(self.fichier_cle):
        self.cle = os.urandom(32)
        with open(self.fichier_cle, "wb") as f_cle:
            f_cle.write(self.cle)
    else:
        with open(self.fichier_cle, "rb") as f_cle:
            self.cle = f_cle.read()

def choisir_dossier(self):
    chemin = QFileDialog.getExistingDirectory(self, "Sélectionner un dossier")
    if chemin:
        self.chemin_cible = chemin
        nom_dossier = os.path.basename(chemin)
        self.label_info.setText(f"<b>Dossier prêt :</b> {nom_dossier}")
        self.label_info.setStyleSheet("color: #2c3e50;")
        self.progress_bar.setValue(0)
        self.label_progression.setText("")

def lancer_travail(self, action):
    if not self.chemin_cible:
```

```
        QMessageBox.warning(self, "Erreur", "Veuillez d'abord choisir un dossier.")
        return

    # On verrouille l'interface
    self.btn_chiffrer.setEnabled(False)
    self.btn_dechiffrer.setEnabled(False)
    self.btn_choisir.setEnabled(False)
    self.progress_bar.setValue(0)

    # Création et configuration du Thread
    self.worker = WorkerCryptage(self.chemin_cible, self.cle, self.taille_chunk, action)

    # Connexion des signaux du Thread vers l'interface graphique
    self.worker.progression.connect(self.progress_bar.setValue)
    self.worker.info_fichier.connect(self.label_progression.setText)
    self.worker.termine.connect(self.travail_termine)

    # Lancement du Thread
    self.worker.start()

def travail_termine(self, fichiers_traites, erreurs):
```

```
        # On déverrouille l'interface
        self.btn_chiffrer.setEnabled(True)
        self.btn_dechiffrer.setEnabled(True)
        self.btn_choisir.setEnabled(True)
        self.label_progression.setText("Opération terminée.")

        # On nettoie le thread
        self.worker.deleteLater()

        QMessageBox.information(self, "Terminé", f"{fichiers_traites} fichiers traités.\n{erreurs} erreurs rencontrées.")

if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = LogicielCryptage()
    fenetre.show()
    sys.exit(app.exec())
```

Ce qui a changé :

1. **Création de la classe** WorkerCryptage **:** Toute la logique de chiffrement/déchiffrement et les boucles os.walk ont été déplacées ici.
2. **Utilisation de** pyqtSignal **:** C'est le cœur du système. Le Worker n'a pas le droit de modifier l'interface (la barre) directement. Il "émet" des signaux (le pourcentage, le nom du fichier en cours), et la fenêtre principale les écoute pour se mettre à jour.
3. **Fusion des fonctions :** Au lieu d'avoir chiffrer_dossier et dechiffrer_dossier dans la classe principale, on a une seule fonction lancer_travail(action) qui démarre le thread en lui disant quoi faire.

4. **Suppression de** QApplication.processEvents() **:** Ce n'est plus du tout nécessaire puisque le travail bloquant est sorti du thread principal. L'interface restera 100% fluide, même avec un fichier de 50 Go.

Voulez-vous que l'on s'attaque ensuite à **l'intégrité des données** en remplaçant ChaCha20 par ChaCha20Poly1305 pour que votre application détecte immédiatement les fichiers corrompus lors du déchiffrement ?

Vous avez dit

Comment créer un exécutable WINDOWS .exe ?

Gemini a dit

Pour transformer votre script Python (avec PyQt6 et Cryptography) en un véritable exécutable Windows (.exe) autonome, l'outil le plus standard et le plus performant est **PyInstaller**.

L'avantage de PyInstaller est qu'il va "embarquer" Python, PyQt6, et toutes vos bibliothèques directement dans l'exécutable. L'utilisateur final n'aura même pas besoin d'avoir Python installé sur sa machine !

Voici la marche à suivre étape par étape :

Étape 1 : Installer PyInstaller

Ouvrez votre terminal ou invite de commandes (cmd/PowerShell) et tapez :

Bash

```
pip install pyinstaller
```

Étape 2 : Préparer la compilation

Dans votre terminal, naviguez jusqu'au dossier où se trouve votre script Python (disons qu'il s'appelle secure_folder.py).

Bash

```
cd chemin\vers\votre\dossier
```

Étape 3 : Lancer la création de l'exécutable

Pour une application graphique comme la vôtre, il faut utiliser des options spécifiques pour cacher la console noire de Windows et tout regrouper dans un seul fichier.

Tapez cette commande :

Bash

```
pyinstaller --onefile --windowed secure_folder.py
```

Que font ces options ?

- --onefile (ou -F) : Demande à PyInstaller de regrouper tous les fichiers, bibliothèques et dépendances dans un **seul et unique fichier .exe**.
- --windowed (ou -w) : Indique qu'il s'agit d'une application graphique (GUI). Cela empêche la fenêtre noire de la console Windows (cmd) de s'ouvrir en arrière-plan quand on lance le logiciel.

(Optionnel) Si vous avez une icône .ico pour votre logiciel, vous pouvez l'ajouter à la commande :

Bash

```
pyinstaller --onefile --windowed --icon=mon_icone.ico secure_folder.py
```

Étape 4 : Récupérer votre .exe

Une fois que PyInstaller a terminé (cela peut prendre une minute ou deux), regardez dans votre dossier. Vous verrez de nouveaux dossiers : build et dist, ainsi qu'un fichier .spec.

- Allez dans le dossier dist (pour *distribution*).
- Vous y trouverez votre fichier secure_folder.exe !
- Vous pouvez maintenant déplacer ce fichier où vous voulez, l'envoyer à des amis, il fonctionnera tout seul. Les dossiers build et le fichier .spec peuvent être supprimés en toute sécurité, ils ne servent qu'à la compilation.

💡 **Deux astuces importantes :**

1. **Le poids du fichier :** PyQt6 et Cryptography sont de grosses bibliothèques. Ne soyez pas surpris si votre .exe fait entre 30 et 60 Mo. C'est tout à fait normal pour une application Python compilée en un seul fichier.
2. **Faux positifs d'Antivirus :** Parfois, Windows Defender ou d'autres antivirus bloquent les .exe créés avec PyInstaller --onefile parce qu'ils n'ont pas de "certificat numérique" officiel (qui coûte cher). Si cela arrive, il suffit d'ajouter une exception dans votre antivirus.

Voulez-vous que l'on intègre la **gestion d'un mot de passe** (avec dérivation de clé robuste) avant de compiler votre application définitive ?

Vous avez dit

D:\PYTHON DJANGO HOPES\LOGICIEL PyQt COURS\Program Python MARIO\CRYPTAGE DOSSIER\chiffrement symétrique Fernet ChaCha20\ChaCha20_pur_UN_FICHIER_peut_DEPASSER_256_Go.py

Gemini a dit

Parfait ! Avec ce chemin exact, voici la commande précise que vous devez taper pour créer votre exécutable .exe.

Puisque votre chemin contient des espaces (comme dans LOGICIEL PyQt COURS ou Program Python MARIO), il est très important de mettre des **guillemets** autour du chemin, sinon Windows ne comprendra pas la commande.

Voici les deux méthodes pour le faire dans votre terminal (Invite de commandes ou PowerShell) :

Méthode 1 : Se déplacer dans le dossier d'abord (Recommandé)

Ouvrez votre terminal et tapez ces trois commandes l'une après l'autre, en appuyant sur Entrée à chaque fois :

1. **Changer de disque (pour passer sur le disque D) :**

Bash

```
D:
```

2. **Aller dans le bon dossier (n'oubliez pas les guillemets) :**

Bash

```
cd "D:\PYTHON DJANGO HOPES\LOGICIEL PyQt COURS\Program Python MARIO\CRYPTAGE DOSSIER\chiffrement symétrique Fernet ChaCha20"
```

3. **Lancer la création de l'exécutable :**

Bash

pyinstaller --onefile --windowed ChaCha20_pur_UN_FICHIER_peut_DEPASSER_256_Go.py

Méthode 2 : Tout faire en une seule ligne

Si vous préférez ne pas naviguer dans les dossiers, vous pouvez donner le chemin complet directement à PyInstaller (toujours avec les guillemets) :

Bash

pyinstaller --onefile --windowed "D:\PYTHON DJANGO HOPES\LOGICIEL PyQt COURS\Program Python MARIO\CRYPTAGE DOSSIER\chiffrement symétrique Fernet ChaCha20\ChaCha20_pur_UN_FICHIER_peut_DEPASSER_256_Go.py"

Une fois la commande terminée, vous trouverez un nouveau dossier nommé dist à l'intérieur de votre dossier chiffrement symétrique Fernet ChaCha20. Votre fichier ChaCha20_pur_UN_FICHIER_peut_DEPASSER_256_Go.exe se trouvera à l'intérieur !

Souhaitez-vous générer l'exécutable tel quel, ou voulez-vous que l'on intègre la **demande d'un mot de passe** au lancement du logiciel avant de le figer en .exe ?

Gemini est une IA et peut se tromper.